https://chatgpt.com/share/6af75b77-15bc-83ea-a9e8-4aa4c715fa8b

https://chatgpt.com/share/6af75b21-06e4-83e9-8956-695945b7145b

https://chatgpt.com/share/6af75ba3-5d04-83ea-9568-58acb1058110

Provide guidance on how to code the project step by step. I'll provide what I have written and then I'd like feedback on what to change/improve.

First step-defining the dataclasses:

```
from dataclasses import dataclass
from drafter import *
from bakery import assert_equal

@dataclass
class State:
    total_instructions: list[list[CrochetInstructions]]
    current_row:int
    total_rows:int
    current_instructions:str
    special_instructions: SpecialInstructions

@dataclass
class CrochetInstructions:
    row: int
    type_of_stitch:str
    stitch_count:int

@dataclass
```

class SafetyEyes:

size:int

row:int

has_safety_eyes:bool

@dataclass

class SpecialInstructions:

safety_eyes:SafetyEyes

stuffing: bool

…

Great! We'll build this step by step, and I'll give you feedback on your code before you move to the next stage. I'll focus on helping you understand the design decisions rather than rewriting the whole project for you.

Step 1: Review your dataclasses

You're off to a good start. You've identified several important concepts in your application: the overall program state, individual crochet rows, and optional special instructions.

Here is my feedback on what to keep and what to reconsider.

1. CrochetInstructions — good starting point

@dataclass

class CrochetInstructions:

row: int

type_of_stitch: str

stitch_count: int

This is a reasonable representation of one row in a crochet pattern.

row: identifies the row number.

type_of_stitch: stores the stitch used.

stitch_count: stores the number of stitches.

Question to consider: Does stitch_count mean the number of stitches the user should make in that row, or the total number of stitches they should have when the row is finished? Those can mean different things, so you should decide what it represents.

2. SafetyEyes — consider whether you need a separate dataclass

@dataclass

class SafetyEyes:

    size: int

    row: int

    has_safety_eyes: bool

This is useful if your website specifically generates patterns for stuffed crochet projects, such as amigurumi toys.

However, consider these questions:

Is the user designing a toy that needs safety eyes, or could they be making any kind of crochet project?

If has_safety_eyes is False, do you still need to store the eye size and row?

Would a simpler Boolean field be enough for your first version?

For an introductory project, you should only track details that your website actually uses.

3. SpecialInstructions — a reasonable idea, but watch the scope

```
@dataclass
class SpecialInstructions:
    safety_eyes: SafetyEyes
    stuffing: bool
```

This gives you a way to represent special project requirements. The potential issue is that your SafetyEyes object is required even when the project doesn't use safety eyes.

Think about how you could represent the absence of safety eyes. Python's None and optional values may be useful here, but first decide whether you need this complexity in your initial version.

Also, stuffing is relevant to some crochet projects but not all. You could keep it if you intend to focus on stuffed toys, or leave it out if your website is meant for general crochet patterns.

4. State — the most important class to reconsider

```
@dataclass
class State:
    total_instructions: list[list[CrochetInstructions]]
    current_row: int
    total_rows: int
    current_instructions: str
    special_instructions: SpecialInstructions
```

In Drafter, the State dataclass holds the information your application needs to remember between user interactions.

There are three things to think about here.

A. Why is total_instructions a nested list?

Your type annotation is:

list[list[CrochetInstructions]]

This means a list containing other lists of crochet instructions. For example, each inner list might represent a separate pattern or project.

Ask yourself: Will your website support multiple patterns, or will it create one pattern at a time?

If you are building one pattern at a time, a single list of CrochetInstructions objects may be sufficient.

If you want multiple patterns, the nested list might make sense, but you would need to define what each inner list represents.

B. Do you need both current_row and total_rows?

These fields aren't necessarily redundant. current_row could track the row the user is working on, while total_rows could track how many rows the pattern contains.

But if the website simply collects row instructions and organizes them, you may not need a separate current-row tracker at all.

C. What is current_instructions for?

A string can be useful for temporarily holding instructions typed by the user or generated by the application. Decide whether this is temporary input or part of the saved pattern.

If it is already represented by an object in total_instructions, you may be storing the same information twice.

5. One Python detail to check

Your State class refers to CrochetInstructions and SpecialInstructions before those classes are defined. Python type annotations can sometimes support forward references, but for a beginner project, the simplest approach is to define the supporting dataclasses before State.

That way, the classes are defined before you refer to them, and the code is easier to follow.

Your next task

Before changing any code, answer these three questions. They'll help us settle the data model before you start building the Drafter interface.

1. What kind of crochet projects will your website support?

General crochet projects

Stuffed toys / amigurumi

2. How many patterns will the user work on at once?

One pattern at a time

Multiple patterns

3. What is the main goal of your first version?

Generate instructions from structured inputs

Let users enter and organize their own row instructions

Track the current row and stitch count

Once we've clarified these choices, we'll revise your dataclasses together. After that, we can move on to initializing the state and creating your first Drafter page.

I want the user to be able to do both stuffed patterns and general patterns. I put the bool in there so that if its false, it wont be added to the final instructions. I want one pattern to be generated but i put it as a nested list because each row will have its own list of instructions. the current instructions will be converting the list I'll eventually loop through and put that into actual sentences

That clarification helps a lot! Your design makes more sense now that I understand how you intend to generate the pattern. I would keep most of your original approach and make a few targeted improvements rather than redesigning everything.

Let's go through your reasoning one point at a time.

1. Supporting both general and stuffed crochet patterns

Your SafetyEyes and SpecialInstructions dataclasses can support this. Your idea of using a Boolean to decide whether safety-eye instructions should be included is reasonable.

```
@dataclass
class SafetyEyes:
    size: int
    row: int
    has_safety_eyes: bool
```

One thing to consider: if has_safety_eyes is False, the size and row don't matter. That's not necessarily a problem, but your pattern-generation code will need to check the Boolean before using those values.

Conceptually, your logic could look like this:

```
if special_instructions.safety_eyes.has_safety_eyes:
    # Add safety-eye instructions to the pattern
```

You don't need to write the entire generation process yet. Just keep in mind that optional instructions should be controlled by conditions like this.

2. One pattern with multiple lists of instructions

Your explanation of the nested list is reasonable: you want each row to contain its own list of instructions.

For example, imagine a pattern with two rows:

Row 1

Instruction 1: Chain 6 stitches

Instruction 2: Join into a ring

Row 2

Instruction 1: Make 6 single crochets

Instruction 2: Increase to 12 stitches

Your nested list would allow you to represent this structure. However, there's one important distinction to make.

Your current annotation is:

total_instructions: list[list[CrochetInstructions]]

This means each inner list contains CrochetInstructions objects. Since each object already has a row field, you could technically store multiple instruction objects for the same row.

My question is: Does each instruction need its own stitch type and stitch count, or does each row have one stitch type and count, with a list of smaller instructions inside it?

If each row has its own group of instructions, your current structure may work. If each row is just one CrochetInstructions object, a flat list would be simpler. The key is to make the data structure match what you want to represent.

3. Using current_instructions to generate sentences

This makes sense as a processing step. You want to take structured data and turn it into readable text.

For example:

Structured input: row=2, type_of_stitch="single crochet", stitch_count=12

Generated sentence: Row 2: Make 12 single crochet stitches.

Your current_instructions: str could hold the resulting sentence or sentences.

One design consideration: if you're generating a complete pattern, you may eventually want to combine all the row sentences into one final pattern string. You don't necessarily need to store every intermediate string in State; you can generate them when needed. For now, your approach is fine as a starting point.

4. What I would change in your dataclasses

I wouldn't rewrite them yet. I'd make just two decisions first:

Clarify the nested list structure. Decide whether each row contains multiple CrochetInstructions objects or whether each row is itself one object with a list of smaller instructions.

Decide what current_row tracks. Is it the row currently being entered in the form, or the row currently being processed by your generation loop? That determines whether it belongs in State.

Everything else can wait until you've started implementing the application.

Your next step

Let's settle the nested list question before moving forward.

When a user designs Row 3, what do you imagine they will enter?

For example, will they enter several separate instructions such as “chain 2,” “make 6 single crochets,” and “increase twice” for that one row? Or will they enter just one stitch type and one stitch count for Row 3?

That answer will help us choose the simplest data structure that still supports your idea.

they will enter seperate instructions like 2 singles, 3 increases, 1 decrease, etc.

That makes the nested list approach much clearer! Since each row can contain several different instructions, I think your overall design is on the right track. Let's refine how the data will be represented before moving on to the Drafter interface.

1. How your data structure will work

Imagine a user creates this pattern:

Example crochet pattern

Row 1

1 instruction

Chain 6

Row 2

3 instructions

2 single crochets

3 increases

1 decrease

Your nested list represents this hierarchy:

The outer list contains all the rows.

Each inner list contains the instructions for one row.

Each instruction needs its own data.

That is a good reason to use a nested list.

2. One important change to consider

Your current CrochetInstructions dataclass is:

```
@dataclass
class CrochetInstructions:
    row: int
    type_of_stitch: str
    stitch_count: int
```

Since you want instructions such as "2 single crochets," "3 increases," and "1 decrease," consider what each object needs to represent.

For example, you could interpret the fields as follows:

Field

Meaning

row

The row the instruction belongs to

type_of_stitch

Single crochet, increase, decrease, etc.

stitch_count

How many times to perform that instruction

Notice that stitch_count would mean the number of times to perform the instruction, not the total stitches in the finished row.

You should decide whether that is the meaning you want. If so, your existing dataclass can already represent these instructions without needing additional fields.

One further consideration: because each inner list represents a row, you might not need to repeat the row number in every instruction object. However, keeping it could make the objects easier to understand when you process them. Either choice can work.

3. What I'd do with your State dataclass

Your current field:

total_instructions: list[list[CrochetInstructions]]

fits your intended structure. I would keep it for now.

Your next task is to think about how a user will add instructions to the current row. For example, when they enter "3 increases," your program needs to create a CrochetInstructions object and add it to the appropriate inner list.

You don't need to code that yet. First, let's settle one more design decision.

How will the user indicate that they have finished entering instructions for a row?

A. They click an "Add Instruction" button for each instruction, then click "Finish Row" when done.

B. They enter a fixed number of instructions for each row, such as three instruction fields.

C. They enter instructions in a text box, and your program tries to interpret the text.

I'd recommend A. It works naturally with your nested list, supports as many instructions per row as needed, and is manageable with Drafter and basic Python.

I think it might make the most sense to do a textbox because a pattern could be 1 sc, 1 inc repeat for the rest of the round. They probably dont want to add a bunch of instructions over and over again. could A be able to do that

Yes! Option A can absolutely support that, and I think there's a good middle ground between a textbox and adding individual instructions one at a time.

The key is to let the user enter a complete set of instructions for a row, including repeats, rather than forcing them to add every stitch separately.

My recommendation: A textbox for each row

For example, your Drafter website could look something like this:

Create a crochet row

Row number

3

Instructions for this row

1 sc, 1 inc, repeat around

Enter crochet abbreviations and repeat instructions in your own words.

Expected total stitches (optional)

e.g. 18

Pattern preview

Row 3: 1 sc, 1 inc, repeat around.

Illustrative mock-up of the interface, not a working pattern-saving form.

The user types their instructions once, and your program stores them. You don't need to interpret every crochet abbreviation or calculate every repeat for the first version.

How this affects your dataclasses

This changes your original design slightly, because you would be storing a complete text instruction for each row rather than breaking every instruction into its own object.

For example, consider these two approaches:

Approach 1: Structured instructions

Instruction 1: single crochet, count 1

Instruction 2: increase, count 1

Repeat information stored separately

This makes it easier to calculate stitches automatically, but the logic becomes more complicated.

Approach 2: User-written instructions

- Row 3: "1 sc, 1 inc, repeat around"

This is much simpler. Python can store the text as a string and include it in the final pattern without needing to understand its meaning.

For your introductory project, I recommend starting with Approach 2.

What would change in your code?

Your existing CrochetInstructions dataclass could be simplified to represent one row containing its own written instructions. Think about which fields you still need:

The row number

The instruction text

Possibly the expected stitch count

You could then store a list of these row objects in State, rather than a nested list. Each object would represent one complete row, even if that row contains multiple stitch instructions.

Your SpecialInstructions dataclass could remain separate for optional details like safety eyes and stuffing, which your program can add to the final output when applicable.

One important limitation

A textbox approach means your website can organize and format the user's instructions, but it won't automatically know whether "1 sc, 1 inc, repeat around" produces the correct number of stitches. That would require additional parsing and crochet-specific logic, which you can leave for a later version.

Your next coding step

Before changing the dataclasses, decide one thing:

Do you want the website to simply organize the user's written instructions into a polished pattern, or should it also generate some instructions automatically from dropdowns and numbers?

I'd recommend organizing written instructions first. Once that works, you can add automatic generation for a few common cases without making the whole project too complicated.

I just want it to store the users input to generate a pattern

Perfect! That makes your project much more manageable. Your website will act as a crochet pattern organizer and formatter: the user writes their own instructions, and your program organizes them into a complete, readable pattern.

You won't need to interpret crochet abbreviations, calculate repeats, or determine whether a pattern is technically correct.

Step 1: Revise your dataclasses

Based on what we've discussed, here's what I'd like you to consider changing in your original code.

1. CrochetInstructions

Each object should represent one complete row, with the user's written instructions stored as text.

Your original fields were row, type_of_stitch, and stitch_count. Ask yourself whether you still need type_of_stitch and stitch_count if the user is typing something like "1 sc, 1 inc, repeat around".

You will likely need:

A row number.

A string containing the user's complete instructions.

You could also retain an expected stitch count if you want users to record it, but it isn't required for formatting their pattern.

2. State

Your total_instructions field can now be a list of CrochetInstructions objects, because each object represents one row. You no longer need a nested list if each row's instructions are stored together in one string.

Think about whether you still need current_row, total_rows, and current_instructions. Keep fields only if they serve a distinct purpose in your program.

3. SpecialInstructions and SafetyEyes

You can keep these for optional details such as safety eyes and stuffing. Your program can append those details to the final pattern only when the user indicates they apply.

For general crochet projects, the special-instructions section can simply be left empty or disabled.

Step 2: Your first coding task

Before moving on to the Drafter page, try revising just your dataclasses.

Use these questions as your checklist:

Does each CrochetInstructions object represent one row?

Can it store an entire row of user-written text?

Does State store a list of those row objects?

Have you removed fields that no longer serve a clear purpose?

Can special instructions be optional?

Send me your revised code when you're ready. I'll review it and explain any changes I'd recommend before we move on to initializing your state and building the first page in Drafter.

I have current row because I will eventually have two routes that wil allow them to go back to a row and update or delete. I used total rows so that the user knows how many rows there are

That makes sense! Both fields have a purpose in your design, so I wouldn't remove them just to simplify the dataclass. Let's distinguish what each one does and how you can use it.

1. current_row — keep it

Your plan is to let users navigate back to an existing row to update or delete it. current_row can track which row the user is currently viewing or editing.

For example:

The user is viewing Row 4.

They click Edit Row.

Your program uses current_row to identify the row being edited.

After saving, the updated instructions replace the previous instructions.

One thing to consider later: will current_row store the row's number (such as 4) or its position in your list (such as index 3)? Python lists start at index 0, so keeping these concepts separate will help prevent bugs.

2. total_rows — useful, but consider how you maintain it

You want users to know how many rows their pattern contains. That's a reasonable feature.

For example, your interface could display:

My Crochet Pattern

Total rows: 8

Row 4 of 8

However, if total_instructions already contains all the rows, you can calculate the total with len(total_instructions).

That means you have two options:

Calculate it when needed: Use len(total_instructions). This avoids the possibility of the stored total becoming incorrect after a row is deleted.

Store it in State: Keep total_rows, but make sure you update it whenever a row is added or deleted.

For your first version, I'd recommend calculating it when needed. You can still display the total to the user without storing a separate field.

3. What about current_instructions?

I'd keep this field if it serves as the temporary text in the form while the user adds or edits a row.

For example, the user types "1 sc, 1 inc, repeat around" into a textbox. current_instructions can hold that text before the user saves the row.

If you use it for that purpose, it has a clear role separate from the saved list of rows.

Your next step

You don't need to change your entire design. For now, I'd suggest:

Keep current_row.

Calculate the total row count using len() instead of storing total_rows.

Keep current_instructions for the text being entered or edited.

Change CrochetInstructions so it stores a row number and a complete instruction string.

Keep your optional special-instructions classes if you intend to include safety eyes and stuffing in the generated pattern.

Now try updating your dataclasses based on these decisions and send me the revised code. I'll review it before we move on to the next step.

```
from dataclasses import dataclass
from bakery import assert_equal
from drafter import *



@dataclass
class CrochetInstructions:
    row: int
    complete_row_instructions:str
@dataclass
class SafetyEyes:
    size:int
    row:int
    has_safety_eyes:bool

@dataclass
class SpecialInstructions:
    safety_eyes:SafetyEyes
```

```
    stuffing: bool


@dataclass
class State:
    total_instructions: list[CrochetInstructions]
    current_row:int
    current_instructions:str
    special_instructions: SpecialInstructions
…
```

This is a solid revision! Your dataclasses now reflect the way you've decided the website will work: each row stores the user's written instructions, and State keeps track of the pattern and the current editing information.

Let's review a few details before moving on.

1. What you've done well

CrochetInstructions — now represents one complete row, which matches your textbox approach.

total_instructions: list[CrochetInstructions] — correctly represents a list of row objects. You no longer need a nested list because each row's instructions are stored in one string.

current_row — supports your plan to navigate to a row for editing or deleting.

current_instructions — can hold the text currently being entered or edited before it is saved.

SpecialInstructions — keeps optional project details separate from the main row instructions.

2. Two things to improve

A. Make special instructions optional

Currently, every SpecialInstructions object requires a SafetyEyes object, even when the user isn't making a stuffed toy or doesn't need safety eyes.

You have two reasonable choices:

Keep a has_safety_eyes Boolean and check it before including the details in the final pattern.

Allow the safety-eye information to be absent when it isn't needed.

For your current skill level, your existing Boolean approach is perfectly acceptable. You can leave it as it is for now. Just remember that the program must check the Boolean before adding safety-eye instructions.

B. Decide what happens when a user edits a row

Your current CrochetInstructions class stores the row number and the complete instructions, which is all you need for the data itself.

Later, when a user edits a row, you'll need to find the matching object in total_instructions and update it. When they delete a row, you'll need to remove the matching object.

One important detail: if a row is deleted, will the remaining rows automatically be renumbered, or will the user manually adjust the row numbers? I'd recommend renumbering them automatically so the final pattern stays organized.

3. A small code-quality suggestion

Your code is valid in structure, but I'd add spaces around the type annotations for readability. For example:

row: int

complete_row_instructions: str

This doesn't change how Python works; it just makes your code easier to read.

You can also remove assert_equal for now if you're not writing tests yet. You can bring it back when you start testing functions.

4. Next step: Initialize your State

Your dataclasses define what information the application stores. The next step is to create the initial State object that Drafter will use when the website first loads.

Before you write it, think about the initial values:

Field

Initial value to consider

total_instructions

An empty list

current_row

A starting row or placeholder value

current_instructions

An empty string

special_instructions

A SpecialInstructions object with initial values

Your next task: Try writing the line that creates your initial State object using State(...). You don't need to build any pages or routes yet.

Send me that line, and I'll check whether the initial values match your dataclasses before we move on to the first Drafter page.

as in start_server(State())?

Almost! start_server(State()) is the right idea, but State() won't work yet because your dataclass requires four fields, and you haven't given them initial values.

Remember, your State dataclass currently has:

```
@dataclass
class State:
    total_instructions: list[CrochetInstructions]
    current_row: int
    current_instructions: str
    special_instructions: SpecialInstructions
```

When you create a State object, Python needs an initial value for each of these fields.

For example, think about what values you would use for:

total_instructions — should the pattern start with any rows?

current_row — what row should the user start on?

current_instructions — should there be any text before the user types something?

special_instructions — what objects need to be created to initialize this field?

Your turn: Try writing State(...) with the appropriate initial values inside the parentheses. You can also initialize it in a variable first, then pass that variable to start_server().

Hint: an empty list is [], an empty string is "", and an integer can start at 0 or 1, depending on your design. The trickiest field is special_instructions, because it requires you to create an instance of another dataclass first.

start_server(State([],0,"",None))

You're very close! Your first three values are reasonable, but there's one thing to fix: None for special_instructions.

Your State dataclass expects a SpecialInstructions object. Python will allow you to pass None when creating the object, but later, if your code tries to access something like state.special_instructions.stuffing, it will raise an error.

You have two options:

Create a SpecialInstructions object with initial values for its fields, including a SafetyEyes object.

Allow special_instructions to be optional by changing its type annotation to indicate that it can be None. This is useful if the user hasn't entered any special instructions yet.

For your project, I'd lean toward option 1 if you want the state to always contain special-instruction settings.

Your turn: Your SafetyEyes dataclass requires three values, and SpecialInstructions requires two fields. Try creating those objects inside your State(...) call.

One other detail: starting current_row at 0 is valid if you use it to mean "no row selected yet." If you want the user to start on Row 1, use 1 instead.

the user can input special instructions but they also dont have to

That makes sense! In that case, None is actually a reasonable choice when the user hasn't provided any special instructions. You just need to make your State dataclass reflect that possibility.

1. Update your State type annotation

Change this field:

special_instructions: SpecialInstructions

to:

special_instructions: SpecialInstructions | None

The | None means the field can contain either a SpecialInstructions object or None.

2. Your initial state

With that change, your original initialization is valid:

start_server(State([], 0, "", None))

Here's what each value represents:

Value

Purpose

[]

No rows have been added yet

0

No row is currently selected

""

No instructions have been entered yet

None

No special instructions have been provided

3. One important thing to remember

Later, when generating the final pattern, check whether special instructions exist before accessing their fields.

For example, conceptually:

```
if state.special_instructions is not None:
    # Include the special instructions
```

You'll also need to decide how the user enters special instructions and how those values get stored, but that can come later.

Next step: Your dataclasses and initial state are now ready for the first Drafter page. I'd suggest building a simple page that displays the pattern title and lets the user enter their first row of instructions. Once that works, we can add the ability to save multiple rows.

is there another way to do the |None it was not something in my course

Yes! Since | None hasn't been covered in your course, there's a simpler alternative that uses concepts you likely already know.

Option 1: Keep SpecialInstructions as a required object (recommended)

Instead of using None when there are no special instructions, create a SpecialInstructions object with default values indicating that the user doesn't need any.

For example, your existing dataclasses could stay the same, and your initial state could look like this:

```
start_server(State(
    [],
    0,
    "",
    SpecialInstructions(
        SafetyEyes(0, 0, False),
```

```
        False
    )
))
```

Here's what those values mean:

SafetyEyes(0, 0, False) — no safety eyes are being used.

The second False — stuffing is not needed.

You still have a SpecialInstructions object, but its settings indicate that neither special feature is selected.

This avoids introducing | None and means you won't have to check whether the entire object exists before accessing its fields.

Option 2: Remove special instructions from the initial version

If you're worried about complexity, you could leave special instructions out of the first version and add them after the main pattern-generation workflow works.

That would let you focus on the core features first: entering rows, saving them, editing them, deleting them, and generating the final formatted pattern.

My recommendation: Use Option 1. It fits your current dataclasses and uses concepts you're already working with. Just remember that 0 is acting as a placeholder for the safety-eye size and row, so your program should only use those values when has_safety_eyes is True.

okay. moving onto my first route: index

```
@route
def index(state:State)->Page:
    return Page(state,[
        Header("Crochet Pattern Builder"),
        "What would you like to do?\n",
        Button("Add a Row","stitch_instructions"),
        Button("Update a Row","update_instructions"),
        Button("Delete a Row","delete_instructions"),
        Button("Insert Pattern Info","pattern_info"),
        Button("Insert Additional Instructions","additional_instructions"),
        Button("View Generated Instructions","final_crochet_instructions")])
```

my goal for this one is to display the users options as to what they want to do as buttons and when they click one og those buttons it takes them to a page with the appropiate textbox/input that they will add

…

Your index route is a good start! You've identified the main actions a user might want to take, and using buttons to navigate to separate pages is a sensible structure for your Drafter application.

Let's review what you've written and what to do next without jumping ahead and writing the whole project for you.

1. What you're doing correctly

```
@route
def index(state: State) -> Page:
```

@route registers the function as a route in Drafter.

state: State gives the page access to your application's shared state.

-> Page indicates that the function returns a Drafter page.

Your Page layout is also appropriate for your goal:

Header gives the page a title.

The text tells users what to do.

Each Button provides a navigation option.

2. How your buttons work

For example:

Button("Add a Row", "stitch_instructions")

The first argument is the text displayed on the button. The second argument is the destination route.

When the user clicks Add a Row, Drafter navigates to the route named stitch_instructions.

That means you will eventually need routes with matching names for all six buttons:

Button

Destination route

Add a Row

stitch_instructions

Update a Row

update_instructions

Delete a Row

delete_instructions

Insert Pattern Info

pattern_info

Insert Additional Instructions

additional_instructions

View Generated Instructions

final_crochet_instructions

You don't need to implement all six immediately. You can build and test them one at a time.

3. One small improvement Your current text is: "What would you like to do?\n",

This is fine as text, but you may find it easier to control the page layout by letting Drafter's components handle the spacing. You can leave it as is for now and adjust the appearance once the routes work.

Your six buttons are also reasonable, although Insert Additional Instructions may eventually be clearer as Add Special Instructions, since that matches your SpecialInstructions dataclass. 4. What to do next: Build the first destination route I recommend implementing stitch_instructions first, since adding rows is the central function of your application.

That page will need to:

1. Ask the user for a row number.
2. Provide a textbox for the complete row instructions, such as 1 sc, 1 inc, repeat around.
3. Include a button to save the row.
4. Provide a way to return to the index page. You will also need to decide how the save button communicates with your program. In Drafter, a button that submits form data is different from a button that simply navigates to another route, so make sure you use the appropriate input and submission components for the version of Drafter you're using.

Your next task: Try writing the stitch_instructions route with a textbox and a button. Don't worry about saving the row yet. First, focus on displaying the inputs and getting the page navigation working.

Send me that code when you're ready, and I'll review it before we add the saving logic.

```
@route

def stitch_instructions(state:State,instructions:CrochetInstructions)->Page:

   return Page(state,[

      Header("Input Row/Stitch Details"),

      Textbox("What Row Are You Working On?"),

      Textbox("What Type and Amount of Stitches Did You Use?"),

      Button("Save","save_and_return")

       ])
```

I know im missing the parts where i need to save the instructions to the various state attributes but this is the start of the route I have. can you guide me on how to draft the rest of the function and make sure what I already have makes sense

You're on the right track! Your stitch_instructions route has the right overall purpose: collect the row number and the user's written crochet instructions, then let them save the row.

The main thing to work on now is how Drafter passes input values into your route and how those values get saved into State. Let's build that logic step by step rather than jumping straight to the finished function.

1. Review your current route

Your code:

```
@route
def stitch_instructions(state: State, instructions: CrochetInstructions) -> Page:
    return Page(state, [
        Header("Input Row/Stitch Details"),
        Textbox("What Row Are You Working On?"),
        Textbox("What Type and Amount of Stitches Did You Use?"),
        Button("Save", "save_and_return")
    ])
```

Here's what I'd change or investigate.

state: State — keep this. It gives you access to the rows already saved in the pattern.

instructions: CrochetInstructions — you probably don't need this parameter on the input page. You're collecting information to create a CrochetInstructions object, so you don't need to pass an already-created object into the form just to make a new one.

The two textboxes — conceptually correct, but each needs a way for Drafter to identify its input and pass the value to the appropriate route.

The Save button — the destination route needs to receive the submitted values, create the row object, and add it to state.total_instructions.

One important detail: the exact arguments supported by Textbox and Button depend on the Drafter component API used in your course. Follow the syntax from your course examples rather than guessing at parameter names.

2. Give each textbox a name Your program needs to distinguish the two values the user enters. In Drafter, the input's name is how you connect it to a parameter in the route that handles the submission.

Conceptually, you want: Textbox promptInput nameData you need What row are you working on?rowRow number What type and amount of stitches did you use?complete_row_instructionsWritten instructions

Check your course's examples for the exact Textbox syntax, but the important idea is that each textbox needs a distinct name that matches the parameter you intend to receive. 3. Separate displaying the form from saving the row Think of this as two jobs. Route 1: stitch_instructions

Displays the two textboxes and Save button.

Route 2: save_and_return

Receives the values and creates a CrochetInstructions object.

Update the state

Adds the object to the list and returns to the main page.

This separation is useful because the first route only needs to display the form. The second route does the actual processing. 4. Plan the saving logic Your save_and_return route will need to perform these steps in order:

1. Receive the row number and instruction text from the form.
2. Create a CrochetInstructions object using those two values.
3. Add that object to state.total_instructions.
4. Return the user to the main page, preserving the updated state. For example, the object creation will follow this general structure: new_row = CrochetInstructions(row=..., complete_row_instructions=...)

The ... represents the values received from the form. Your next task is to work out how those values will be passed into the route.

One other detail: if your row number comes from a regular text input, it may arrive as a string. Make sure it is converted to an integer before you create the object, if necessary.

5. Your next coding task

Try revising stitch_instructions first:

- Remove the instructions: CrochetInstructions parameter for now.
- Give each textbox a distinct input name.
- Keep the Save button pointing to save_and_return.

Then write the function header for save_and_return, including state and the two values the form will submit. Don't worry about the body yet.

Send me those two pieces of code, and I'll check the route parameters and input connections before you implement the saving logic.

```
@route
def stitch_instructions(state:State)->Page:
 return Page(state,[
 Header("Input Row/Stitch Details"),
 Textbox("What Row Are You Working On?", state.current_row),
 Textbox("What Type and Amount of Stitches Did You Use?", state.total_instructions),
 Button("Save","save_and_return")
 ])
```

updated route

Good progress! You removed the extra instructions: CrochetInstructions parameter, so this route is now focused on displaying the form. There are two things to adjust before you work on saving.

1. The Textbox arguments

Textbox("What Row Are You Working On?", state.current_row)Textbox("What Type and Amount of Stitches Did You Use?", state.total_instructions)

The second argument is not necessarily the place to put a state value. In Drafter, you need to connect each textbox to a form input name so the save route can receive the values the user types.

- Row textbox: should collect the row number the user is entering.

- Instructions textbox: should collect the complete row instructions as a string, such as "1 sc, 1 inc, repeat around".

Your state.current_row is for tracking the current row in your app, and state.total_instructions is a list of saved row objects. Neither should be used as the instructions textbox's initial value.

2. Keep the form inputs separate from saved data

When the user clicks Save, your save_and_return route should receive the submitted row number and instruction text. It can then create a CrochetInstructions object and append it to state.total_instructions.

Your next step is to check the textbox examples from your course materials so you use the correct Drafter syntax for naming inputs. The exact syntax can vary by the version or examples your course uses.

One question: Have you used named Textbox inputs in a Drafter assignment before? If so, show me one of those examples, and we can match its syntax here.

Continued from Crochet Pattern Generator Features what is a form input for textboxes I havent used these A form input is just the information a user types into a textbox on your webpage. You haven't missed anything — it's a way of connecting what the user types to the Python function that handles the Save button.

For your crochet app, imagine this flow:

1. The user fills in the form

What row are you working on?

5 What stitches did you use?

1 sc, 1 inc, repeat around Save

2. Drafter sends those values to your save route

Row number: "5"

Instructions: "1 sc, 1 inc, repeat around"

3. Your Python function saves the information

It can create a CrochetInstructions object and add it to state.total_instructions.

How does this relate to your code? Your current line is:

Textbox("What Row Are You Working On?", state.current_row)

You're thinking about what value belongs in the textbox. That's reasonable! But the label and the input's name serve different purposes:

The label tells the user what to type.

The input name identifies the value so your save route can receive it.

The value the user types is the actual data you want to save.

The exact syntax for naming a Textbox depends on the Drafter version and the conventions your course uses, so I don't want to give you code that might not work in your environment.

Let's take the simplest next step: do you have an example from class where a Textbox sends information to a route? If you paste it here, I'll help you adapt that pattern to your crochet app.

so its just a name for me to refer back to when I write my save_and_return? Yes, exactly! You've got it. 😊

The input name is a name you use to identify the value the user typed, so Drafter can pass that value to your save_and_return route.

For your crochet app, you might use names like:

"row" — for the row number.

"instructions" — for the written stitch instructions.

Then your save_and_return route would use those same names for its parameters, along with state, so it knows which submitted value is which.

One important distinction: the input name is not the value itself. For example, "row" is the name, while 5 is what the user types.

That way, your save route can take the submitted values, create a CrochetInstructions object, and append it to state.total_instructions.

@route def stitch_instructions(state:State)->Page: return Page(state,[Header("Input Row/Stitch Details"), Textbox("What Row Are You Working On?", "row"), Textbox("What Type and Amount of Stitches Did You Use?", "instructions"), Button("Save","save_and_return")])

Nice! Your route is much closer to what you need. You've given each textbox a distinct name: "row" and "instructions". That makes it easier to connect the form to your save route.

@route def stitch_instructions(state:State)->Page: return Page(state,[Header("Input Row/Stitch Details"), Textbox("What Row Are You Working On?", "row"), Textbox("What Type and Amount of Stitches Did You Use?", "instructions"), Button("Save","save_and_return")])

One thing to verify: Check your course's Drafter examples to confirm whether the second positional argument to Textbox is the input name in your version. The names are sensible; we just need to make sure the syntax matches your library.

Your next step Now try writing the function header only for save_and_return.

It needs to receive:

state: State

The submitted row number

The submitted instructions text

Think about what data type each submitted value should have. The row number may arrive as a string, even though you want to store it as an integer in CrochetInstructions.

Send me your function header, and I'll check it before you write the body.

TextBox(name) TextBox(name, default_value) TextBox(name, default_value, kind) Ah, perfect! That tells us how your version of Drafter's Textbox works.

The syntax is:

Textbox(name) Textbox(name, default_value) Textbox(name, default_value, kind)

So in your code:

Textbox("What Row Are You Working On?", "row")

the first argument is treated as the name, and the second as the default value — meaning "row" would appear as the initial textbox content, not serve as its input name.

Let's switch the order of your thinking. The name comes first, and the optional default value comes second.

For example, based on the syntax you shared, you could use:

Textbox("row") Textbox("instructions")

These give your inputs names without specifying default values.

One thing to check: since the first argument is the name, you may want to use descriptive labels elsewhere on the page so users know what to enter. The exact way to add those labels depends on the Drafter components your class has covered.

Now you can move on to your save_and_return route! Try writing its function header with state, row, and instructions. Remember to think about the types of those values.

@route def stitch_instructions(state:State)->Page: return Page(state,[Header("Input Row/Stitch Details"), "What Row Are You Working On?, TextBox("row"), "Write This Row's Instruction", TextBox("instructions"), Button("Save","save_and_return")])You're getting the structure right! Separating the text labels from the TextBox inputs is a good approach. There are just a couple of small syntax fixes.

1. Fix the quotation marks This line is missing its closing quotation mark:

"What Row Are You Working On?,

It should be:

"What Row Are You Working On?",

2. Your updated route

@route def stitch_instructions(state: State) -> Page: return Page(state, [Header("Input Row/Stitch Details"), "What Row Are You Working On?", TextBox("row"), "Write This Row's Instructions", TextBox("instructions"), Button("Save", "save_and_return")])

This assumes your library uses the TextBox capitalization and syntax from your course examples.

3. What happens next? Your form now has two named inputs:

row — the row number entered by the user.

instructions — the written instructions for that row.

When the user clicks Save, the save_and_return route needs to receive those values.

Your next task: Try writing just the function header for save_and_return. Include state, row, and instructions as parameters, with type hints if you know the appropriate types. I'll check it before you write the function body.

@route

def save_and_return(state:State, row:int,instructions:str)->Page:

    row_instructions = CrochetInstructions

    row_instructions.row = row

    row_instructions.complete_row_instructions = instructions

    state.total_instructions.append(row,instructions)

    return index(state)

You're very close! Your overall plan is right: create a row object, save it in the state's list, and return to the main page. There are two main issues to fix.

1. Creating the CrochetInstructions object

You wrote:

```
row_instructions = CrochetInstructions
row_instructions.row = row
row_instructions.complete_row_instructions = instructions
```

CrochetInstructions is the class itself. You need to create an instance of that class by passing its required fields to it.

Remember your dataclass:

```
@dataclass
class CrochetInstructions:
    row: int
    complete_row_instructions: str
```

Hint: When you create an instance of a dataclass, you use the class name followed by parentheses containing the values for its fields. You can assign the result directly to row_instructions.

How could you pass row and instructions into those parentheses?

2. Adding the object to your list

You wrote:

state.total_instructions.append(row, instructions)

The append() method adds one item to a list at a time. Your list is meant to contain CrochetInstructions objects, not separate row and instruction values.

Once you've created row_instructions, append that single object to the list.

3. Returning to the main page

return index(state)

This part looks good! It calls your index route with the updated state.

Your turn

Try fixing just the object creation and the append() line. Keep your return statement as it is, then send me your revised function. I'll check it before we move on.

@route

def save_and_return(state:State, row:int,instructions:str)->Page:

    row_instructions = CrochetInstructions(row,instructions)

    state.total_instructions.append(row_instructions)

    return index(state)

Perfect! 🎉 You've fixed both issues correctly. Your function now follows the right sequence.

@routedef save_and_return(state: State, row: int, instructions: str) -> Page: row_instructions = CrochetInstructions(row, instructions) assert_equal(index(State([],1, "", SpecialInstructions(SafetyEyes(0,0,False),False))), Page(State([],1, "", SpecialInstructions(SafetyEyes(0,0,False),False))), [Header("Crochet Pattern Builder"), "What would you like to do", Button("Add a Row","stitch_instructions"), Button("Update a Row","update_instructions"),

Button("Delete a Row","delete_instructions"), Button("Insert Pattern Info","pattern_info"), Button("Insert Additional Instructions","additional_instructions"), Button("View Generated Instructions","final_crochet_instructions")]) return index(state)

Here's why each part works:

1. **Create the object:** CrochetInstructions(row, instructions) creates a new object with the row number and its instructions.
2. **Save it:** .append(row_instructions) adds that object to state.total_instructions.
3. **Return to the main page:** index(state) displays the main page with the updated state.

One thing to test

Try adding two different rows and see whether both appear in state.total_instructions. Also, verify that your TextBox("row") input actually provides an integer to the route; if it provides a string, you'll need to convert it before creating the object.

What's next?

Your **Add a Row** workflow is nearly complete! The next step is to make sure the user can see the rows they've saved. At the moment, returning to index(state) doesn't display the saved instructions unless you add that functionality to the page.

I'd recommend adding a simple list of saved rows to your main page before moving on to updating and deleting rows.

```
assert_equal(index(State([],1, "", SpecialInstructions(SafetyEyes(0,0,False),False))),
        Page(State([],1, "", SpecialInstructions(SafetyEyes(0,0,False),False))), [
            Header("Crochet Pattern Builder"),
            "What would you like to do",
            Button("Add a Row","stitch_instructions"),
            Button("Update a Row","update_instructions"),
            Button("Delete a Row","delete_instructions"),
            Button("Insert Pattern Info","pattern_info"),
            Button("Insert Additional Instructions","additional_instructions"),
```

```
            Button("View Generated Instructions","final_crochet_instructions")
        ])
```

my assert equal test for index. im getting an error

I see a couple of things to check! The most likely issue is the structure of your assert_equal call.

Your test currently has this structure:

```
assert_equal(
    index(State(...)),
    Page(State(...)),
    [Header(...), ...]
)
```

But assert_equal expects two arguments: the actual result and the expected result. Right now, you're passing three.

The expected Page should contain both the state and the list of page elements.

Try organizing your test like this:

```
assert_equal(
    index(State([], 1, "", SpecialInstructions(SafetyEyes(0, 0, False), False))),
    Page(
        State([], 1, "", SpecialInstructions(SafetyEyes(0, 0, False), False)),
        [
            Header("Crochet Pattern Builder"),
```

```
            "What would you like to do?\n",
            Button("Add a Row", "stitch_instructions"),
            Button("Update a Row", "update_instructions"),
            Button("Delete a Row", "delete_instructions"),
            Button("Insert Pattern Info", "pattern_info"),
            Button("Insert Additional Instructions", "additional_instructions"),
            Button("View Generated Instructions", "final_crochet_instructions")
        ]
    )
)
```

Two other things to verify

Text must match exactly. Your original route had "What would you like to do?\n", but your test uses "What would you like to do". The punctuation and newline can matter in an equality test.

Check the error message. If it still fails, paste the exact error. An error about the number of arguments is different from an assertion failure, and the message will help us pinpoint the cause.

One more note: make sure your index route really uses the same text and buttons as the expected Page in your test.

how would i test my stitch instructions route

For your stitch_instructions route, you want to test that it returns the correct Page with the right labels, textboxes, and Save button.

1. Write an assert_equal test

Based on your current route:

@route

```
def stitch_instructions(state: State) -> Page:
    return Page(state, [
        Header("Input Row/Stitch Details"),
        "What Row Are You Working On?",
        TextBox("row"),
        "Write This Row's Instructions",
        TextBox("instructions"),
        Button("Save", "save_and_return")
    ])
```

Your test should follow the same structure as your index test:

```
test_state = State(
    [],
    1,
    "",
    SpecialInstructions(SafetyEyes(0, 0, False), False)
)

assert_equal(
    stitch_instructions(test_state),
    Page(test_state, [
        Header("Input Row/Stitch Details"),
        "What Row Are You Working On?",
        TextBox("row"),
        "Write This Row's Instructions",
        TextBox("instructions"),
```

```
        Button("Save", "save_and_return")
    ])
)
```

2. What does this test check?

Does the route return a Page?

Does the page contain the correct header and labels?

Are both textboxes included?

Does the Save button point to save_and_return?

Does the returned page contain the same state you passed in?

The expected page must match your actual route exactly, including punctuation and component arguments.

3. One important distinction

This test checks that the route displays the form correctly. It does not test whether entering a row and clicking Save actually stores the instructions. Your save_and_return route needs its own test for that behavior.

Run this test and, if it fails, send me the exact error message. We can use it to identify what needs fixing.

```
assert_equal(save_and_return(State([CrochetInstructions(1, "1 sc, 1 inc")], 1, "1 sc, 1 inc",
SpecialInstructions(SafetyEyes(0,0,False),False))),
                    Page(State([CrochetInstructions(1,"1 sc, 1 inc")],1,
                "1 sc, 1 inc",SpecialInstructions(SafetyEyes(0,0,False),False)),
```

[Header("Crochet Pattern Builder"),

"What would you like to do?",

Button("Add a Row", "stitch_instructions"),

Button("Update a Row", "update_instructions"),

Button("Delete a Row", "delete_instructions"),

Button("Insert Pattern Info", "pattern_info"),

Button("Insert Additional Instructions", "additional_instructions"),

Button("View Generated Instructions", "final_crochet_instructions")]))

…

You're on the right track! Your test is checking that save_and_return saves the row and then returns the main page. There are two important things to fix or verify.

1. Your function call is missing two arguments

Your current function header is:

def save_and_return(state: State, row: int, instructions: str) -> Page:

That means your test must pass three arguments:

state

row

instructions

But your test only passes the State object. You need to provide the row number and instruction text separately, because those are the values submitted by the form.

2. Your expected state should include the new row

Your function creates a CrochetInstructions object and appends it to state.total_instructions.

So if your starting state already contains:

[CrochetInstructions(1, "1 sc, 1 inc")]

and you submit row 2 with "2 sc, 1 inc", the resulting list should contain both rows.

That means your test should distinguish between:

Input state: the state before saving.

Expected state: the state after saving.

Your next step

Try revising your test so that it:

Passes state, row, and instructions to save_and_return.

Starts with an empty total_instructions list, to make the expected result easier to check.

Expects the returned page to contain one newly saved row.

One more thing: your expected index page has "What would you like to do?", while your earlier version included a newline ("\n") at the end. Make sure your expected page matches your actual index route exactly.

can you give me an example

Sure! Here's an example of a test for your save_and_return route, using the dataclasses and route structure you've been working with.

The idea is to start with an empty list of instructions, save one row, and check that the returned page contains that row and the main page's buttons.

```
test_state = State(
    [],
    0,
    "",
    SpecialInstructions(SafetyEyes(0, 0, False), False)
)

expected_state = State(
    [CrochetInstructions(2, "1 sc, 1 inc")],
    0,
    "",
    SpecialInstructions(SafetyEyes(0, 0, False), False)
)

assert_equal(
    save_and_return(test_state, 2, "1 sc, 1 inc"),
    Page(expected_state, [
        Header("Crochet Pattern Builder"),
        "What would you like to do?",
        Button("Add a Row", "stitch_instructions"),
        Button("Update a Row", "update_instructions"),
```

```
        Button("Delete a Row", "delete_instructions"),
        Button("Insert Pattern Info", "pattern_info"),
        Button("Insert Additional Instructions", "additional_instructions"),
        Button("View Generated Instructions", "final_crochet_instructions")
    ])
)
```

Why this works

test_state starts with an empty total_instructions list.

save_and_return(test_state, 2, "1 sc, 1 inc") passes all three arguments your function expects.

Your function creates CrochetInstructions(2, "1 sc, 1 inc") and appends it to the list.

expected_state represents the state after saving.

assert_equal compares the actual returned page against the expected page.

Important: This assumes your index route uses exactly the header, text, and buttons shown above. If your route has different punctuation or a newline in the text, match that in the test.

One subtle point: your function appends to the existing state list, so test_state itself may also be modified during the test. That's normal for this implementation.

moving onto the next route:

@route

```
def additional_instructions(state:State)->Page:
    return Page(state,[
        Header("Input Any Additional Details"),
        "Did You Use Safety Eyes?",
        CheckBox("safety_eyes"),
        "Did You Stuff Your Project?",
        CheckBox("stuffing"),
        Button("Save","save_additional_instructions")
         ])
```

I also need to have a box for the to input the size of the eyes would I put an if statement here underneath the checkbox or when i define save_additional_instructions?

I'd put the if statement in additional_instructions, because that controls what the user sees on the page. Your save_additional_instructions route will handle saving the answers afterward.

There are two parts to think about:

Displaying the eye-size box: If the user checks “Did You Use Safety Eyes?”, show a textbox asking for the size.

Saving the answers: In save_additional_instructions, save whether safety eyes were used, their size, and whether the project was stuffed.

How to structure the page

Conceptually, your route would look like this:

```
@route
def additional_instructions(state: State) -> Page:
```

```
    return Page(state, [
        Header("Input Any Additional Details"),
        "Did You Use Safety Eyes?",
        CheckBox("safety_eyes"),

        # If safety eyes are selected, show the size textbox here

        "Did You Stuff Your Project?",
        CheckBox("stuffing"),
        Button("Save", "save_additional_instructions")
    ])
```

The comment marks where your conditional display logic would go. It isn't executable code yet.

One important detail: An if statement only works if your route has access to the checkbox's current value. Check how your version of Drafter handles checkbox inputs and conditional page elements. If the checkbox value only arrives when the user submits the form, you may need a slightly different approach to show the size field dynamically.

My recommendation for your project

Since this is a CS1 project, keep it simple. You could always display the eye-size textbox, with a label like “Safety eye size (if applicable).” Then, in save_additional_instructions, only use the size when has_safety_eyes is True.

That avoids needing a dynamic checkbox interaction and is easier to implement and test.

```
@route
def additional_instructions(state:State)->Page:
    return Page(state,[
```

```
Header("Input Any Additional Details"),
"Did You Use Safety Eyes?",
CheckBox("safety_eyes"),
"Did You Stuff Your Project?",
if "safety_eyes" == True:
   TextBox("eye_size")
CheckBox("stuffing"),
Button("Save","save_additional_instructions")
 ])
```

You're thinking about the logic correctly! You want the eye-size textbox to appear only when safety eyes are selected. The issue is where and how you're writing the if statement.

1. Why your current code doesn't work

You wrote:

if "safety_eyes" == True: TextBox("eye_size")

There are two problems:

- "safety_eyes" is a string (the literal text), so comparing it to True will always be false.
- Python doesn't let you put a normal if statement directly inside a list like that.

2. The simpler approach I'd recommend

Since you're building this in Drafter and want to keep the project manageable, I'd display the eye-size textbox all the time. The user can leave it blank if they aren't using safety eyes.

@routedef additional_instructions(state: State) -> Page: return Page(state, [Header("Input Any Additional Details"), "Did You Use Safety Eyes?", CheckBox("safety_eyes"), "Safety eye size (leave blank if not applicable):", TextBox("eye_size"), "Did You Stuff Your Project?", CheckBox("stuffing"), Button("Save", "save_additional_instructions")])

This avoids conditional display logic and is easier to test.

3. What happens in save_additional_instructions?

That route will receive the submitted values. There, you can check whether safety_eyes is True before saving or using eye_size.

Your next step: Try updating your page to display the eye-size textbox all the time. Then write the function header for save_additional_instructions, including the state and the three submitted values: safety eyes, eye size, and stuffing.

I'd like it so that the box is not always there

That makes sense! You want the eye-size textbox to appear **only when the user checks "Did You Use Safety Eyes?"**. We can work toward that.

The important detail is that a normal Python if statement builds the page when the route runs. To make the textbox appear or disappear as the user checks or unchecks the box, Drafter needs a way to update the page in response to that interaction.

There are two possible approaches:

- **Simpler:** Use a separate route or page step after the user indicates they use safety eyes. That fits the route-based structure you've already been building.
- **More interactive:** Update the page immediately when the checkbox changes. This depends on which interactive features your course's Drafter version supports.

I'd recommend the simpler approach for your CS1 project.

One question: Does your course have an example of a CheckBox that changes what appears on a page, or have you only used checkboxes to submit values through a button so far?

what would it look like to make a seperate route?

A separate route would let you ask whether the user used safety eyes first, then show the eye-size textbox on a second page **only if they answered yes**. This fits the route structure you've already been using.

. Route one: Ask about additional details

Your first route could ask about safety eyes and stuffing, then send the user to a different route depending on their answer.

Conceptually:

```
@routedef additional_instructions(state: State) -> Page:    return Page(state, [        Header("Input Any Additional Details"),        "Did You Use Safety Eyes?",        CheckBox("safety_eyes"),        "Did You Stuff Your Project?",        CheckBox("stuffing"),        Button("Continue", "next_instructions")    ])
```

This displays the first page. The next_instructions route would need to receive the checkbox values when the user clicks Continue.

2. Route two: Decide which page to show

The next route checks the answer and chooses the appropriate page.

```
@routedef next_instructions(state: State, safety_eyes: bool, stuffing: bool) -> Page:    if safety_eyes:        return eye_size_page(state, stuffing)    else:        return save_without_eye_size(state, stuffing)
```

The idea is:

- If safety_eyes is True, show a page that asks for the eye size.
- Otherwise, skip that page and continue saving the additional instructions.

The function names here are examples; you'll need to create the routes and saving logic that match your project.

3. Route three: Ask for the eye size

```
@routedef eye_size_page(state: State, stuffing: bool) -> Page:    return Page(state, [        Header("Safety Eye Details"),        "What size safety eyes did you use?", TextBox("eye_size"),        Button("Save", "save_additional_instructions")    ])
```

This page is only shown if the user answered yes.

One important detail

Your SpecialInstructions dataclass already has a SafetyEyes object containing size, row, and has_safety_eyes, plus a stuffing field. You'll need to make sure your routes preserve the checkbox answers while moving between pages and then update that object when saving.

Also, this example assumes your Drafter version passes checkbox values to route parameters the way your text inputs do. Check your course's checkbox examples for the exact behavior.

I'd build this one step at a time: first get the initial checkbox page working, then make next_instructions receive the checkbox answers correctly. Once that works, add the eye-size page.

ive used helper functions to return values

That's helpful! Since you've used helper functions that return values, you can use the same general idea here. The only new part is that your helper function would return a list of page elements instead of a single value.

However, there's one important limitation: your helper function needs to know whether safety_eyes is checked before it can decide whether to include the eye-size textbox.

For example, the logic inside the helper could be:

```
def additional_instructions_form(safety_eyes: bool):
    elements = [
        Header("Input Any Additional Details"),
        "Did You Use Safety Eyes?",
        CheckBox("safety_eyes")
    ]

    if safety_eyes:
        elements.append("What Size Are Your Safety Eyes?")
        elements.append(TextBox("eye_size"))

    elements.extend([
        "Did You Stuff Your Project?",
        CheckBox("stuffing"),
        Button("Save", "save_additional_instructions")
```

```
    ])

    return elements
```

This demonstrates how to build the page conditionally. It doesn't yet make the checkbox interactive, though. Your route would need to run again with the updated checkbox value to show the eye-size field.

Let's choose the right next step

I'd suggest we first check how your course handles checkbox values. When you click a button after checking a CheckBox("safety_eyes"), does Drafter pass the checked value into the destination route as a parameter?

If you're not sure, we can build on the checkbox route you've already written and test that behavior first.

An unchecked box still submits a value: the parameter receives False, not nothing. (Behind the scenes, Drafter adds a hidden companion field so the unchecked state is submitted too.)

Passing the current state value as default_value keeps the box reflecting what was last saved, as in the example.

Great, that explains how your Drafter checkboxes work! The key detail is that an unchecked box submits False, so your route can make a decision based on the user's answer.

For your project, here's how that fits with a helper function.

1. Your helper function

The helper can decide whether to include the eye-size textbox based on the checkbox value:

def additional_instructions_form(safety_eyes: bool) -> list: elements = [Header("Input Any Additional Details"), "Did You Use Safety Eyes?", CheckBox("safety_eyes", safety_eyes)] if safety_eyes: elements.append("What Size Are Your Safety Eyes?") elements.append(TextBox("eye_size")) elements.extend(["Did You Stuff Your Project?", CheckBox("stuffing"), Button("Save", "save_additional_instructions")]) return elements

The important limitation Your current route could call the helper like this: @routedef additional_instructions(state: State) -> Page: return Page(state, additional_instructions_form(False))

But that always passes False. To make the textbox appear after the user checks the box, something needs to receive the submitted checkbox value and rebuild the page.

A helper function doesn't handle the checkbox click by itself; it only builds the page based on the value it receives.

3. What I recommend

Since you want to avoid adding another route, let's check whether your Drafter version supports having a checkbox submit to the same route or otherwise refresh the page when its value changes.

Do your course notes show an example of a CheckBox that has a route or action attached to it, or does it only show the CheckBox(name, default_value) syntax?

Notice that CheckBox("safety_eyes", safety_eyes) uses the current value as its default, so the checkbox can reflect the value passed to the helper.

```
@route
def save_additional_instructions(state:State, has_eyes: safety_eyes, eye_insert: safety_eye_insert, eye_size: safety_eye_size, has_stuffing:stuffing)-> Page:
    additional_details = SpecialInstructions(SafetyEyes(0,0,False)False)
    if has_eyes:
        additional_details.safety_eyes.has_safety_eyes = True
        additional_details.safety_eyes.size = eye_size
        additional_details.safety_eyes.row = eye_insert
    elif has_stuffing:
        additional_details.stuffing = True
    state.special_instructions = additional_details
    return index(state)
```

You're making good progress! You've got the right idea for save_additional_instructions: create a SpecialInstructions object, fill it based on the user's answers, save it in the state, and return to index.

There are a few syntax and logic issues to fix. Let's go through them one at a time.

1. Fix your function parameters

You wrote:

```
def save_additional_instructions(
    state: State,
    has_eyes: safety_eyes,
    eye_insert: safety_eye_insert,
    eye_size: safety_eye_size,
    has_stuffing: stuffing
) -> Page:
```

In a type hint, the part after the colon should be a data type, such as bool, int, or str — not the name of a textbox or checkbox.

Based on the information you're collecting, think about these types:

Parameter

Likely type

Why

has_eyes

bool

Whether the checkbox is checked

eye_insert

int

The row where the eyes are inserted

eye_size

int

The size of the safety eyes

has_stuffing

bool

Whether the stuffing checkbox is checked

Your exact textbox input names need to match the names you use in your form.

2. Fix the SpecialInstructions constructor

You wrote:

additional_details = SpecialInstructions(SafetyEyes(0,0,False)False)

You're missing a comma between the two arguments. Remember your dataclass structure:

@dataclassclass SpecialInstructions: safety_eyes: SafetyEyes stuffing: bool

It needs two arguments: a SafetyEyes object and a Boolean value for stuffing.

3. Your if statement needs one adjustment

Currently, you have:

if has_eyes: additional_details.safety_eyes.has_safety_eyes = True
additional_details.safety_eyes.size = eye_size additional_details.safety_eyes.row = eye_insertelif has_stuffing: additional_details.stuffing = True

The problem is the elif. If the user uses safety eyes and stuffing, the elif block will be skipped because has_eyes is already true.

Use two separate if statements so both answers can be saved independently.

Also, consider initializing the SafetyEyes object with has_eyes rather than always starting it as False and changing it afterward. Either approach can work.

4. Saving to state

These two lines are on the right track:

These two lines are on the right track:

state.special_instructions = additional_detailsreturn index(state)

They save the new special instructions in the state and return the user to the main page.

Your next step

Try revising your function with these three fixes:

1. Correct the parameter type hints.
2. Fix the SpecialInstructions constructor.
3. Replace elif with a separate if.

Send me your revised function, and I'll check it. One additional detail to keep in mind: your SafetyEyes dataclass includes the row where the eyes are inserted, so your form will need an input for that as well as the eye size.

```
@route
def save_additional_instructions(state:State, safety_eyes:bool, safety_eye_insert:int,
safety_eye_size:int, stuffing:bool)-> Page:
 additional_details = SpecialInstructions(SafetyEyes(0,0,False),False)
 if safety_eyes:
 additional_details.safety_eyes.has_safety_eyes = True
 additional_details.safety_eyes.size = safety_eye_size
 additional_details.safety_eyes.row = safety_eye_insert
 if stuffing:
 additional_details.stuffing = True
 state.special_instructions = additional_details
 return index(state)
```

This is looking much better! Your function now has the correct parameter types, uses two independent if statements, and saves the result to state.special_instructions. Nice work.

Your current function:

Two things to check

1. Make sure your form input names match your route parameters.

Your form needs inputs named exactly safety_eyes, safety_eye_insert, safety_eye_size, and stuffing so Drafter can pass the submitted values to this route. The eye row and size inputs need to be included in the form when you use them.

2. Consider simplifying the initialization.

Right now, you initialize the safety-eye details to SafetyEyes(0, 0, False) and then change the fields if the user selected safety eyes. That's valid and consistent with your current dataclasses.

One behavior to think about: if the user doesn't use safety eyes, the size and row stay at 0, and has_safety_eyes stays False. That's a sensible default for your app.

Next step: Test it

I recommend testing the case where both safety eyes and stuffing are True. That verifies your two separate if statements preserve both choices.

Then test the case where both are False and check that the defaults are saved correctly.

Your route's main logic looks sound. The remaining thing to verify is that the form passes all four values correctly.

assert_equal(save_additional_instructions(State([],0,"",SpecialInstructions(SafetyEyes(4,8,True), False))),

Page(State([], 0, "",

SpecialInstructions(SafetyEyes(4,8,True),False)),

[Header("Crochet Pattern Builder"),

"What would you like to do?",

Button("Add a Row", "stitch_instructions"),

Button("Update a Row", "update_instructions"),

Button("Delete a Row", "delete_instructions"),

Button("Insert Pattern Info", "pattern_info"),

Button("Insert Additional Instructions", "additional_instructions"),

Button("View Generated Instructions", "final_crochet_instructions")]))

assert_equal(save_addtional_instructions(State([],0,"",SpecialInstructions(SafetyEyes(0,0,False),
True))),

Page(State([CrochetInstructions(1,"1 sc 1 inc")],1,

"",SpecialInstructions(SafetyEyes(0,0,False),False)),

[Header("Crochet Pattern Builder"),

"What would you like to do?",

Button("Add a Row", "stitch_instructions"),

Button("Update a Row", "update_instructions"),

Button("Delete a Row", "delete_instructions"),

Button("Insert Pattern Info", "pattern_info"),

Button("Insert Additional Instructions", "additional_instructions"),

Button("View Generated Instructions", "final_crochet_instructions")]))

…

You're on the right track using assert_equal, but both tests need a few changes to match your save_additional_instructions function.

1. Pass all required arguments

Your function expects five arguments:

```
save_additional_instructions(
    state,
    safety_eyes,
    safety_eye_insert,
    safety_eye_size,
    stuffing
)
```

Your tests currently pass only the state, so Python will report that required arguments are missing. You need to supply values for each parameter.

2. First test: safety eyes are already in the state

Your function does not read the existing special instructions to decide what to save. Instead, it creates a new SpecialInstructions object from the submitted checkbox and textbox values.

For a test where safety eyes are selected and stuffing is not selected, you could start with:

```
test_state = State(
    [],
    0,
    "",
    SpecialInstructions(SafetyEyes(0, 0, False), False)
)

assert_equal(
    save_additional_instructions(test_state, True, 8, 4, False),
    Page(
        State(
            [],
            0,
            "",
            SpecialInstructions(SafetyEyes(4, 8, True), False)
        ),
        [
            Header("Crochet Pattern Builder"),
            "What would you like to do?\n",
```

```
            Button("Add a Row", "stitch_instructions"),
            Button("Update a Row", "update_instructions"),
            Button("Delete a Row", "delete_instructions"),
            Button("Insert Pattern Info", "pattern_info"),
            Button("Insert Additional Instructions", "additional_instructions"),
            Button("View Generated Instructions", "final_crochet_instructions")
        ]
    )
)
```

This assumes the arguments are ordered as state, safety_eyes, safety_eye_insert, safety_eye_size, stuffing, and that your index page contains the newline shown in the earlier version.

3. Second test: stuffing is selected Your second expected result currently has a row in total_instructions, but your save_additional_instructions function doesn't add or remove rows. It only updates special_instructions.

So use an empty list of rows in this test too, and expect stuffing to be True: test_state = State([], 0, "", SpecialInstructions(SafetyEyes(0, 0, False), False))assert_equal(save_additional_instructions(test_state, False, 0, 0, True), Page(State([], 0, "", SpecialInstructions(SafetyEyes(0, 0, False), True)), [Header("Crochet Pattern Builder"), "What would you like to do?\n", Button("Add a Row", "stitch_instructions"), Button("Update a Row", "update_instructions"), Button("Delete a Row", "delete_instructions"), Button("Insert Pattern Info", "pattern_info"), Button("Insert Additional Instructions", "additional_instructions"), Button("View Generated Instructions", "final_crochet_instructions")]))

Before you run the tests

- Fix the spelling in save_addtional_instructions if your actual function is named save_additional_instructions.
- Make sure the expected Page matches your actual index function exactly, including \n in the greeting if it is present.
- These tests check the two cases separately. Once they pass, consider a third test where both safety_eyes and stuffing are True to check that your two independent if statements work together.

```
#initial additional_instructions
assert_equal(additional_instructions(State([],0,"",SpecialInstructions(SafetyEyes(0,0,False),False
))),
   Page(State([],0,"",SpecialInstructions(SafetyEyes(0,0,False),False)),
      Header("Input Any Additional Details"),
      "Did You Use Safety Eyes?",
      CheckBox("safety_eyes"),
      "Safety Eye Size in mm (Leave blank if not used)",
      TextBox("safety_eye_size"),
      "What Row Did You Add Your Eyes?",
      TextBox("safety_eye_insert"),
      "Did You Stuff Your Project?",
      CheckBox("stuffing"),
      Button("Save","save_additional_instructions")))
```

Your test is very close! The main issue is that Page should receive the list of page elements as its second argument, so wrap the components in square brackets. Also, your test should match the actual additional_instructions route exactly.

```
assert_equal(
   additional_instructions(
      State([], 0, "", SpecialInstructions(SafetyEyes(0, 0, False), False))
   ),
   Page(
      State([], 0, "", SpecialInstructions(SafetyEyes(0, 0, False), False)),
      [
         Header("Input Any Additional Details"),
         "Did You Use Safety Eyes?",
         CheckBox("safety_eyes"),
```

```
            "Safety Eye Size in mm (Leave blank if not used)",
            TextBox("safety_eye_size"),
            "What Row Did You Add Your Eyes?",
            TextBox("safety_eye_insert"),
            "Did You Stuff Your Project?",
            CheckBox("stuffing"),
            Button("Save", "save_additional_instructions")
        ]
    )
)
```

What this test checks:

The route returns a Page.

The page keeps the original state unchanged.

The page includes the safety-eyes checkbox, size and row inputs, stuffing checkbox, and Save button in the correct order.

One important detail: this test checks that the size and row textboxes are displayed even when the safety-eyes checkbox is unchecked. That matches the route you wrote. If you later want those textboxes to appear only when safety eyes are selected, you'll need to add conditional page-building behavior.

next two routes. I change my state class and added a dataclass because i didnt have anywhere to store the pattern details

@dataclass

```
class Pattern:
    hook_size: int
    yarn_name: str
    yarn_type: str

@dataclass
class State:
    total_instructions: list[CrochetInstructions]
    current_row: int
    current_instructions: str
    special_instructions: SpecialInstructions
    pattern_details = Pattern

@route
def pattern_info(state:State)->Page:
    return Page(state,[
        Header("Input Pattern Details"),
        "Hook Size (mm)",
        Textbox("hook_size"),
        "Yarn Brand",
        TextBox("yarn_brand"),
        "Yarn Name",
        TextBox("yarn_name"),
        Button("Save","save_pattern_info")
         ])

@route save_pattern_info(state:State, hook_size:int, yarn_brand:str, yarn_name:str)->Page:
```

```
    pattern =  Pattern(0,"","")
    pattern.hook_size = hook_size
    pattern.yarn_name = yarn_name
    pattern.yarn_type = yarn_type
    state.pattern_details = pattern
    return index(state)
```

…

Good change adding a Pattern dataclass! You're separating the general pattern details from the row instructions and special instructions, which makes your State easier to organize. There are a few things to fix in these two routes.

1. Fix your State dataclass

This line isn't creating a field that stores a Pattern object:

pattern_details = Pattern

Instead, add a type annotation so each State has a pattern_details field:

@dataclassclass State: total_instructions: list[CrochetInstructions] current_row: int current_instructions: str special_instructions: SpecialInstructions pattern_details: Pattern

Now, when you initialize your app with start_server, you'll need to provide an initial Pattern object too. For example:

Pattern(0, "", "")

2. Fix pattern_info

Your form is nearly correct. Two things to notice:

- You named the dataclass field yarn_type, but your form asks for yarn_brand and yarn_name. Decide what information you want to store. If you want both the brand and yarn name, you'll need fields for both.

- You wrote Textbox("hook_size") with a lowercase b. If your course uses TextBox, capitalize it consistently.

For example, if you want to store hook size, yarn brand, and yarn name, your dataclass could be:

@dataclass

class Pattern:

    hook_size: int

    yarn_brand: str

    yarn_name: str

That matches the information your form asks the user to enter.

3. Fix save_pattern_info

There are three issues in your current route:

1. The route decorator needs parentheses: @route.

2. Your function accepts yarn_name, but then uses yarn_type, which isn't a parameter.

3. You can create the Pattern directly from the submitted values instead of creating a placeholder and changing each field afterward.

The structure should be:

@routedef save_pattern_info(state: State,   hook_size: int,   yarn_brand: str,   yarn_name: str) -> Page: pattern = Pattern(hook_size, yarn_brand, yarn_name)   state.pattern_details = pattern   return index(state)

This version assumes you've updated Pattern to have the three fields shown above. The form's textbox names match the route parameters, so Drafter knows which submitted value belongs to each parameter.

4. One test to write next

Test that saving pattern details actually updates state.pattern_details and returns to the home page. Use a test state initialized with an empty Pattern(0, "", ""), then call save_pattern_info with sample values and check that the returned Page contains the updated pattern details.

One design question: Do you want to store the yarn's brand and name separately, or do you want the yarn's name and type? Your current dataclass and form describe different sets of information, so it's best to settle that before writing the test.

```
#Project has safety eyes
assert_equal(save_additional_instructions(initial_test_state,True,4,8,False),
        Page(State([], 0, "",
        SpecialInstructions(SafetyEyes(8,4,True),False),Pattern(0,"","")),
            [Header("Crochet Pattern Builder"),
             "What would you like to do?",
             Button("Add a Row", "stitch_instructions"),
             Button("Update a Row", "update_instructions"),
             Button("Delete a Row", "delete_instructions"),
             Button("Insert Pattern Info", "pattern_info"),
             Button("Insert Additional Instructions", "additional_instructions"),
             Button("View Generated Instructions", "final_crochet_instructions")]))

#Project has stuffing
assert_equal(save_additional_instructions(initial_test_state,False,0,0,True),
        Page(State([], 0, "",
        SpecialInstructions(SafetyEyes(0,0,False),True),Pattern(0,"","")),
```

```
        [Header("Crochet Pattern Builder"),
         "What would you like to do?",
         Button("Add a Row", "stitch_instructions"),
         Button("Update a Row", "update_instructions"),
         Button("Delete a Row", "delete_instructions"),
         Button("Insert Pattern Info", "pattern_info"),
         Button("Insert Additional Instructions", "additional_instructions"),
         Button("View Generated Instructions", "final_crochet_instructions")]))
```

i put in a variable for the initial state since it was getting really long. I'm getting an error in my assert equal tests now for these two.

…

Using initial_test_state is a good idea! It makes your tests shorter. I suspect the error is related to how your new Pattern field was added to State, but let's narrow it down.

1. Check your initial_test_state

Make sure it includes all five fields in the correct order:

```
initial_test_state = State(
    [],
    0,
    "",
    SpecialInstructions(SafetyEyes(0, 0, False), False),
    Pattern(0, "", "")
)
```

2. Check whether your test state is being modified

Your save_additional_instructions route changes state.special_instructions directly. If you use the same initial_test_state for both tests, the first call changes that object. The second test may then start with the results of the first test instead of a fresh state.

Try creating a new state for each test:

```
eyes_test_state = State(
    [],
    0,
    "",
    SpecialInstructions(SafetyEyes(0, 0, False), False),
    Pattern(0, "", "")
)
```

Use eyes_test_state in the first test, and create a separate stuffing_test_state for the second.

3. Check the expected page Your expected State values look consistent with the two scenarios:

- Safety eyes: SafetyEyes(8, 4, True) and stuffing is False.
- Stuffing: SafetyEyes(0, 0, False) and stuffing is True. Also make sure the actual index route includes exactly the same text and buttons as your expected Page, including whether "What would you like to do?" ends with a newline. Can you share the exact error? If those changes don't fix it, paste the full error message from Python. In particular, whether it says TypeError, AssertionError, or something else will tell us what to investigate next.

it just says i got too many tiems (1>0) and expected 0 but got 1 state current_row

That error is a useful clue! It sounds like your test is comparing the returned Page against the expected Page, and the State objects differ in current_row.

The message says the actual state has current_row = 1, but your expected state has current_row = 0 (or vice versa). How to fix it Check the value of current_row in initial_test_state. For example,

if you defined it like this: initial_test_state = State([], 1, # current_row is 1 here "", SpecialInstructions(SafetyEyes(0, 0, False), False), Pattern(0, "", ""))

then your expected Page must also use 1 for current_row, not 0.

Alternatively, if current_row should start at 0, change it in initial_test_state to 0.

Important: Your save_additional_instructions route shouldn't change current_row at all. So the expected state should preserve whatever value was in the state passed to the route.

Can you paste your initial_test_state definition? Then we can identify exactly which value needs to change.

initial_test_state = State([],0,"",

SpecialInstructions(SafetyEyes(0,0,False),False),

Pattern(0,"","",""))

Your initial_test_state looks correct — current_row starts at 0. So the issue is likely that your save_additional_instructions function or one of your tests is reusing a state that has been modified.

Your function assigns state.special_instructions, but it shouldn't change current_row. Let's isolate the two tests by giving each one its own fresh state.

```
# Project has safety eyes
eyes_test_state = State(
    [], 0, "",
    SpecialInstructions(SafetyEyes(0, 0, False), False),
    Pattern(0, "", "")
)

assert_equal(
    save_additional_instructions(eyes_test_state, True, 4, 8, False),
    Page(
        State(
```

```
            [], 0, "",
            SpecialInstructions(SafetyEyes(8, 4, True), False),
            Pattern(0, "", "")
        ),
        [
            Header("Crochet Pattern Builder"),
            "What would you like to do?",
            Button("Add a Row", "stitch_instructions"),
            Button("Update a Row", "update_instructions"),
            Button("Delete a Row", "delete_instructions"),
            Button("Insert Pattern Info", "pattern_info"),
            Button("Insert Additional Instructions", "additional_instructions"),
            Button("View Generated Instructions", "final_crochet_instructions")
        ]
    )
)
```

Make the same change for the stuffing test, using a separate stuffing_test_state.

If it still fails

Please paste your current save_additional_instructions function and the exact error text. The important clue is that the error says current_row is 1 when the expected value is 0. Since the initial state you showed has current_row = 0, we need to find where that value becomes 1 or where the test's actual state comes from.

i'm now making helper functions that will build the strings that are going to be called in the route to build the final generated pattern. Starting with final_addtional_details

```
def final_additional_details(state:State)->str:
    final_additional_details = ""
    if state.special_instructions.safety_eyes.has_safety_eyes == True and
state.special_instructions.stuffing == True :
        final_additional_details = str("Place" + state.special_instructions.safety_eyes.size + "Safety
Eyes in Row" + state.special_instructions.safety_eyes.row + "\n" + "Stuff Project")
    elif state.special_instructions.safety_eyes.has_safety_eyes == True and
state.special_instructions.stuffing == False :
        final_additional_details = str("Place" + state.special_instructions.safety_eyes.size + "Safety
Eyes in Row" + state.special_instructions.safety_eyes.row + "\n")
    elif state.special_instructions.safety_eyes.has_safety_eyes == False and
state.special_instructions.stuffing == True :
        final_additional_details = str("Stuff Project")
    else:
        final = ""
…
```

This is a good approach! Having final_additional_details as a helper function keeps the route that generates the final pattern cleaner. Your if/elif structure covers all four combinations of safety eyes and stuffing. There are just a few Python issues to fix.

1. Convert numbers to strings

Your size and row fields are integers. Python can't concatenate an integer directly with a string.

For example, this will cause a TypeError:

```
"Place" + state.special_instructions.safety_eyes.size
```

You can use str() on the individual numbers:

```
"Place " + str(state.special_instructions.safety_eyes.size)
```

Notice that I also added a space after "Place" so the words don't run together.

2. Fix the final else

You currently have:

else: final = ""

But your function's string variable is called final_additional_details, not final. Change the assignment so the variable is consistent. Otherwise, the function won't return the expected empty string in that case.

3. Make sure your function returns a string

Your function is annotated with -> str, which means it should return a string. At the moment, none of your branches has a return statement, so the function returns None by default.

You could simplify your function by building the result in pieces, rather than repeating the safety-eye text in two branches.

Here's a structure to consider:

```
def final_additional_details(state: State) -> str:
    final_additional_details = ""

    if state.special_instructions.safety_eyes.has_safety_eyes:
        final_additional_details = (
            "Place "
            + str(state.special_instructions.safety_eyes.size)
```

```
            + " mm Safety Eyes in Row "
            + str(state.special_instructions.safety_eyes.row)
        )


    if state.special_instructions.stuffing:
        if final_additional_details != "":
            final_additional_details += "\n"
        final_additional_details += "Stuff Project"


    return final_additional_details
```

This version uses two independent if statements because safety eyes and stuffing can both be included. It also avoids adding an unnecessary newline when only one detail is present.

4. Test the four combinations

You should test all four possibilities:

Safety eyes	Stuffing	Expected result
No	No	""
Yes	No	Safety-eye details
No	Yes	"Stuff Project"
Yes	Yes	Safety-eye details followed by "Stuff Project" on a new line

For example, if the safety eyes are size 8 and are inserted in row 4, the combined result should be:

Place 8 mm Safety Eyes in Row 4

Stuff Project

I'd recommend testing the helper function directly with assert_equal before connecting it to your final pattern route. That way, if the generated text is incorrect, you know the problem is in the helper function rather than the route.

def final_additional_details(state:State)->str:

    final_additional_details = ""

    if state.special_instructions.safety_eyes.has_safety_eyes == True and state.special_instructions.stuffing == True :

        final_additional_details = "Place " + str(state.special_instructions.safety_eyes.size) + "mm " + "Safety Eyes in Row " + str(state.special_instructions.safety_eyes.row + "\n" + "Stuff Project")

    elif state.special_instructions.safety_eyes.has_safety_eyes == True and state.special_instructions.stuffing == False :

```
        final_additional_details = str("Place " + state.special_instructions.safety_eyes.size + "mm
"+ "Safety Eyes in Row " + state.special_instructions.safety_eyes.row)

    elif state.special_instructions.safety_eyes.has_safety_eyes == False and
state.special_instructions.stuffing == True :

        final_additional_details = str("Stuff Project")

    else:

        final_additional_details = ""

    return final_additional_details


#Eyes and Stuffing

assert_equal(final_additional_details(State([],0,0,SpecialInstructions((8,4,True),True),
Pattern(0,"","")))

            ,"Place 8mm Safety Eyes in Row 4")


if state.special_instructions.safety_eyes.has_safety_eyes == True and
state.special_instructions.stuffing == True :

AttributeError: 'tuple' object has no attribute 'has_safety_eyes'
```

The error is coming from your test setup, not from the if statement itself! You're passing a tuple where your code expects a SafetyEyes object.

1. Fix the test state

Currently, you have:

SpecialInstructions((8, 4, True), True)

The parentheses (8, 4, True) create a tuple. Your function then tries to access .has_safety_eyes on that tuple, which causes the AttributeError.

Instead, create the proper dataclass object:

SpecialInstructions(SafetyEyes(8, 4, True), True)

Your corrected test setup becomes:

assert_equal(final_additional_details(State([], 0, "", SpecialInstructions(SafetyEyes(8, 4, True), True), Pattern(0, "", ""))), "Place 8mm Safety Eyes in Row 4\nStuff Project")

2. Fix two issues in your function

There are still two string-concatenation issues to address.

Issue A: The row number and newline are inside the same str() call.

In your first if branch, this part:

str(state.special_instructions.safety_eyes.row + "\n" + "Stuff Project")

tries to add an integer row number to strings before converting it. Convert the row number to a string first:

str(state.special_instructions.safety_eyes.row) + "\n" + "Stuff Project"

def final_pattern_details(state:State)->str:

final_pattern_details = ""

if state.pattern_details.hook_size =! 0 and state.pattern_details.yarn_type =! "" and state.pattern_details.yarn_name =! "":

final_pattern_details = "Hook size: " str(state.pattern_details.hook_size) + "\n" + "Yarn Used: " + state.pattern_details.yarn_type + state.pattern_details.yarn_name

if state.pattern_details.hook_size == 0 and state.pattern_details.yarn_type =! "" and state.pattern_details.yarn_name =! "":

"Yarn Used: " + state.pattern_details.yarn_type + state.pattern_details.yarn_name

if

return final_pattern_details

how should i work this one without making an if statement for each case

You can avoid an if statement for every possible combination by building the output string in pieces, just like we did with final_additional_details. That way, you only need an if for each piece of information that might be included.

One small syntax note: Python uses != for “not equal,” not =!.

Here's how I'd approach it step by step.

1. Start with an empty string

```
def final_pattern_details(state: State) -> str:
    final_pattern_details = ""
```

2. Add the hook size if it exists

```
    if state.pattern_details.hook_size != 0:
        final_pattern_details = "Hook size: " + str(state.pattern_details.hook_size) + " mm"
```

Notice that we convert the integer to a string using str() before concatenating it.

3. Add yarn information if it exists

Check whether the yarn fields contain values. If they do, add the yarn information to the existing string.

```
    if state.pattern_details.yarn_type != "" or state.pattern_details.yarn_name != "":
        if final_pattern_details != "":
            final_pattern_details += "\n"
        final_pattern_details += "Yarn Used: " + state.pattern_details.yarn_type + " " +
state.pattern_details.yarn_name
```

Here, or means the yarn details are included if either field has a value. The nested if adds a newline only when hook information has already been added.

4. Return the finished string

```
    return final_pattern_details
```

How this handles different cases

```
def final_instructions(state:State)->str:
```

```
    total = ""
    for instructions in state.total_instructions:
        total = total + "Row " + str(instructions.row) + ": " +
instructions.complete_row_instructions
    return total
```

this is my last helper function. Using this to iterate through the list of crochet instructions and make that into a string

This is a good use of a for loop! You're iterating through state.total_instructions and combining each row number with its corresponding instructions. There's just one improvement I'd recommend.

Add a newline between rows

Currently, each row is appended immediately after the previous row, so the output might look like this:

Row 1: 6 scRow 2: 1 inc, 6 scRow 3: 8 sc

Add "\n" at the end of each row so the generated pattern is easier to read.

```
def final_instructions(state: State) -> str:
    total = ""
    for instructions in state.total_instructions:
        total = total + "Row " + str(instructions.row) + ": " +
instructions.complete_row_instructions + "\n"
    return total
```

Now the output will look like:

Row 1: 6 sc

Row 2: 1 inc, 6 sc

Row 3: 8 sc

Test your helper function

Try testing it with two rows to confirm that the loop includes both:

```
test_state = State(
    [
        CrochetInstructions(1, "6 sc"),
        CrochetInstructions(2, "1 inc, 6 sc")
    ],
    0,
    "",
    SpecialInstructions(SafetyEyes(0, 0, False), False),
    Pattern(0, "", "")
)

assert_equal(
    final_instructions(test_state),
    "Row 1: 6 sc\nRow 2: 1 inc, 6 sc\n"
)
```

One small detail: this version leaves a newline after the final row. That's usually fine, but if you'd prefer not to have one at the very end, you can handle that later.

Your three helpers now have distinct jobs:

final_pattern_details(state) builds the hook and yarn information.

final_instructions(state) builds the row-by-row instructions.

final_additional_details(state) builds optional safety-eye and stuffing notes.

Those are good building blocks for your final generated-pattern route.

FAILURE at line 310 (assert_equal):

The two values were different:

- Expected 'Row 1: 6 sc in mr\nRow 2: 6 inc\nRow 3:1 inc, 1 sc\n' but got 'Row 1: 6 sc in mr\nRow 2: 6 inc\nRow 3: 1 inc, 1 sc\n'

>>>

i cant find the difference

@route

def final_crochet_instructions(state:State)->Page:

final = ""

final = final + final_pattern_details(state) + final_additional_details(state) + final_instructions(state)

return final

You're very close! Your helper functions are being combined correctly in principle, but there are two things to fix in this route.

1. Return a Page, not a string

Your route is annotated with -> Page, but your last line returns final, which is a string. In Drafter, you need to wrap the generated pattern in a Page.

For example, your route could return:

```
return Page(state, [
    Header("Your Finished Crochet Pattern"),
    final
])
```

2. Add line breaks between sections

Right now, you concatenate the three helper results directly, so the end of one section might run into the next. Add "\n" between them:

```
@route
def final_crochet_instructions(state: State) -> Page:
    final = ""
    final = (
        final_pattern_details(state)
        + "\n"
        + final_additional_details(state)
        + "\n"
        + final_instructions(state)
    )
    return assert_equal(final_crochet_instructions(final_instructions_test1),
            Page(State, [
                Header("Final Crochet Pattern"),
                "Hook Size: 7 mm \n Yarn Used: Yarnsmiths Chenille Chunky\nRow 1: 6 sc in mr\n"
                ]))
    ])
```

One thing to watch out for

If final_additional_details(state) returns an empty string, you may get extra blank lines. That's not necessarily a problem for your project, but you can improve the formatting later if you want.

I'd test this route with a state that has pattern details, at least one row, and no additional instructions, then test it again with safety eyes and stuffing included.

assert_equal(final_crochet_instructions(final_instructions_test1),

Page(State, [

Header("Final Crochet Pattern"),

"Hook Size: 7 mm \n Yarn Used: Yarnsmiths Chenille Chunky\nRow 1: 6 sc in mr\n"

]))

FAILURE at line 336 (assert_equal):

The two values were different:

- In Page state: Expected type type but got type State

The error is because you're using State (the class itself) instead of a State object (an instance of the class) in your expected Page.

You currently have:

Page(State, [

But it should use the state from your test, which appears to be final_instructions_test1:

Page(final_instructions_test1, [

So your test should look like this:

```
assert_equal(
    final_crochet_instructions(final_instructions_test1),
    Page(
        final_instructions_test1,
        [
            Header("Final Crochet Pattern"),
            "Hook Size: 7 mm \n Yarn Used: Yarnsmiths Chenille Chunky\nRow 1: 6 sc in mr\n"
        ]
    )
)
```

One more important thing: keep this assert_equal test outside your final_crochet_instructions function. Your route should only build the final string and return the Page. The test calls the route afterward to check that the returned page matches what you expect.

the next route is this one. How should I approach this? Its updating instructions so I need the same textboxes for stitch instructions except the row one wil be locating what row they want to change and the stitch instructions will replace the current instructions. I'll need another save_update route i believe too.

```
@route
def update_instructions(state:State)->Page:
```

Yes, that's exactly the right approach! Separate it into two routes:

update_instructions displays the form, asking which row to update and what the replacement instructions should be.

save_update finds that row in state.total_instructions, replaces its instructions, and returns to index(state).

The key difference from your save_and_return route is that you're modifying an existing CrochetInstructions object instead of appending a new one.

1. Build the update form

You can reuse the same textbox names and labels from your add-row form, since row identifies the row to update and instructions contains the replacement text.

@route

def update_instructions(state: State) -> Page:

return Page(state, [

Header("Update Row Instructions"),

"Which Row Would You Like to Update?",

TextBox("row"),

"Enter the Replacement Instructions",

TextBox("instructions"),

Button("Save Update", "save_update")

])

2. Plan the save_update route Before writing the code, think through the steps:
- Receive state, row, and instructions as parameters.
- Loop through state.total_instructions.
- Check whether each item's row matches the row number the user entered.
- If it matches, replace that item's complete_row_instructions with the new text.
- Return index(state). Here's a useful reminder from your dataclass: @dataclassclass CrochetInstructions: row: int complete_row_instructions: str

Each item in state.total_instructions has both a row number and its instructions. You only want to change the instructions for the matching row, not its row number. 3. One important decision What should happen if the user enters a row number that doesn't exist?

For a first project, I'd recommend keeping it simple: if no row matches, leave the instructions unchanged and return to the home page. Later, you could add an error message if you want.

Try writing save_update using those steps. Send me your attempt, and I'll help you check the loop and the replacement logic without taking over the whole route.

@route def update_instructions(state:State)->Page: return Page(state, [Header("Update Row Instructions"), "Which Row would you like to update?", TextBox("row"), "Input updated instructions", TextBox("instructions"), Button("Save","save_update")])

@route def save_update(state:State, row:int,instructions:str)->Page: for instruction in state.total_instructions: if row == instruction.row: instruction.complete_row_instructions = instructions else: instruction.complete_row_instructions = instruction.complete_row_instructionsreturn index(state)

This is looking good! Your two routes are structured correctly, and the main logic in save_update is right. You're looping through the rows, checking for a matching row number, and replacing the instructions when you find it.

There are just two things to consider.

1. Remove the unnecessary else You currently have: else: instruction.complete_row_instructions = instruction.complete_row_instructions

This assigns the existing value back to itself, so it doesn't change anything. You can remove the else entirely. If the row doesn't match, the loop simply moves on to the next instruction.

next two routes. this is to delete a row (or an instance of crochetinstructons).

```
@route
def delete_instructions(state:State)->Page:
    return Page(state, [
        Header("Update Row Instructions"),
        "Which Row would you like to delete?",
        TextBox("row"),
        Button("Save","delete_row")])

@route
def delete_row(state:State,row:int)->Page:
    remove_instruction = []
    for instructions in state.total_instructions:
        if instructions.row == row:
```

```
        instructons = []
    return index(state)
```

You're thinking about the right steps! Your delete_instructions form is almost there. The main thing to solve is how to remove the matching CrochetInstructions object from state.total_instructions.

1. Fix the deletion logic

You started with:

```
remove_instruction = []
```

You don't actually need that extra list if you use Python's list .remove() method.

Also, this line in your attempt:

```
instructons = []
```

doesn't remove anything from state.total_instructions. It just reassigns a different variable.

Since you want to remove an item from the list, think about these steps:

Loop through state.total_instructions.

Find the instruction whose row matches the requested row.

Remove that instruction object from state.total_instructions.

Return index(state).

2. One important detail about removing items in a loop Removing items from a list while iterating over that same list can cause problems, because the list changes as the loop progresses.

For your project, a simple approach is to find the matching object first, then remove it after the loop.

You could use a variable to remember the matching object: remove_instruction = None

Inside the loop, if the row matches, assign the object to remove_instruction. After the loop, check whether you found one before removing it.

That way, you avoid modifying the list while you're iterating through it. 3. Correct the page heading Your delete form currently says: Header("Update Row Instructions")

Change that to something like: Header("Delete Row Instructions")

That makes the page's purpose clear to the user. 4. Test it Try creating a state with three rows, then delete row 2. Your expected state should contain row 1 and row 3, with row 2 removed.

Also test a row number that doesn't exist. Your function should leave the list unchanged and return to the home page.

Give the delete_row route another try using the remove_instruction variable, and send me your revised version!

@route def delete_row(state:State,row:int)->Page: remove_row = Nonefor instructions in state.total_instructions: if instructions.row == row: remove_row = row state.total_instruction.remove()return index(state)

You're close! You've correctly started with remove_row = None and are checking whether the row numbers match. There are just three small issues to fix.

1. Store the instruction object, not the row number You currently have:
2. remove_row = row

But row is just an integer. You want to store the matching CrochetInstructions object so you can remove it from the list.

Use:

remove_row = instructions

2. Remove the object from the correct list

Your code has:

state.total_instruction.remove()

Two issues here:

- Your state field is named total_instructions (plural).
- .remove() needs the object you want to remove as an argument.

3. Remove it after the loop

It's safer to finish searching the list before removing an item, rather than changing the list while iterating over it.

Your route's structure should look like this:

@routedef delete_row(state: State, row: int) -> Page: remove_row = None for instructions in state.total_instructions: if instructions.row == row: remove_row = instructions if remove_row is not None: state.total_instructions.remove(remove_row) return index(state)

How it works

- remove_row = None means no matching row has been found yet.
- The loop finds the object with the requested row number.
- After the loop, .remove(remove_row) removes that object from the list.
- If the row doesn't exist, the list stays unchanged.

Testing tip: Try deleting the middle row from a list of three CrochetInstructions objects. Check that only that row disappears and the other two remain.